

Smart Parking & EV Charging

# RAPPORT DE PROJET : Simulation Smart City (C++ & Raylib)

## TITRE DU PROJET :

SIMULATION DE STATIONNEMENT INTELLIGENT  
ET STATION DE RECHARGE VE

### RÉALISÉ PAR :

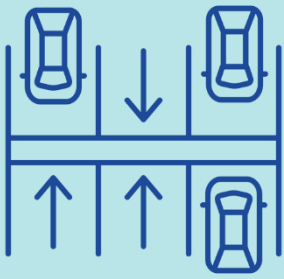
- AYADI YASSIN
- EL KADI MONSSIF
- CHRIAA ZAKARIAE
- ABDENBI MEHDI
- BENMOUSSA MOHAMED REDA

### ENCADRÉ PAR :

*M . IKRAM BEN ABDELOUAHAB*

### ANNÉE UNIVERSITAIRE :

2025 – 2026



# SOMMAIRE :

---

- SOMMAIRE :

1. Introduction Générale
2. Conception et Architecture
3. Logique de Simulation (Zone 1 : Recharge)
4. Le "Handoff" : Transition d'États
5. Gestion Intelligente (Zone 2 : Stationnement)
6. Fonctionnalités Techniques Principales
7. Captures d'écran de la simulation
8. Conclusion

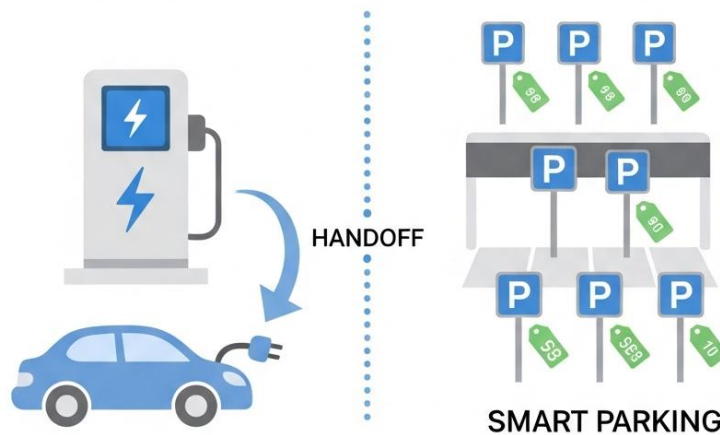
# 1. INTRODUCTION GÉNÉRALE :

La gestion du trafic et des infrastructures urbaines est un enjeu majeur des villes modernes (Smart Cities). Ce projet consiste en la conception et le développement d'une simulation graphique 2D en C++ illustrant un écosystème complexe intégrant deux zones distinctes :

1. Une station de recharge pour véhicules électriques (VE)
2. Un parking intelligent payant.

L'objectif est de simuler le comportement autonome de véhicules qui naviguent, prennent des décisions basées sur leur niveau de batterie et leur budget, et interagissent avec l'environnement.

Le projet met en œuvre une architecture de "State Switch Handoff" (Passage de relais d'état) permettant aux véhicules de conserver leurs données (budget, batterie) lorsqu'ils transitent d'une zone logique à une autre.



---

## Les Outils Utilisés :



- **Langage C++** : Choisi pour sa performance et sa gestion rigoureuse de la mémoire via la Programmation Orientée Objet (POO).



- **Raylib** : Une bibliothèque graphique légère et efficace, idéale pour le prototypage rapide de simulations 2D/3D.



- **Visual Studio Code** : Environnement de développement et gestionnaire de compilation.

## 2. CONCEPTION ET ARCHITECTURE :

### Structure du Projet

Le code a été modulaire et séparé en plusieurs classes pour respecter les principes de responsabilité unique (SRP).

- **Game Loop (Main)** : Orchestre la mise à jour des deux zones et le rendu global.
- **Shared.h** : Ce fichier agit comme une configuration globale. Il contient les constantes (dimensions de l'écran, vitesse des voitures, tarifs) et les structures de données partagées.
- **Car.h** et **SmartCar.h** : Ces fichiers définissent les "agents".
- **Zone 1 (Charging)** : Gérée par la classe EVStation.
- **Zone 2 (Parking)** : Gérée par la classe ParkingManager.

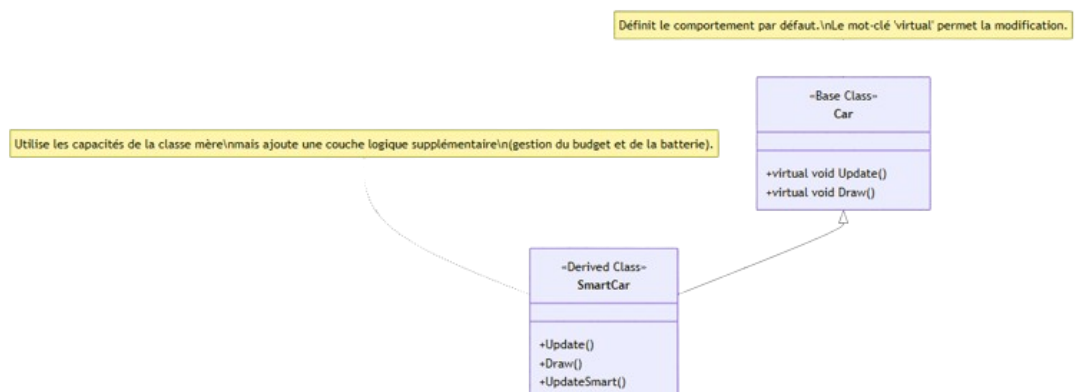
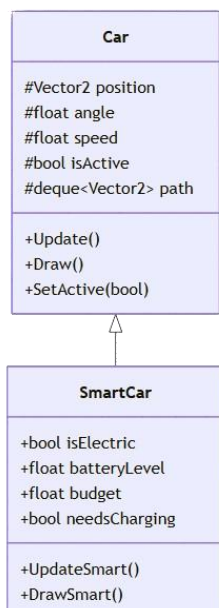
### Paradigme Utilisé

Nous avons utilisé la **Programmation Orientée Objet (POO)** avec un accent fort sur :

#### 1. L'Héritage

: ChargingCar et ParkingCar se rvent de base, tandis que SmartChargingCar et SmartParkingCar étendent les fonctionnalités (gestion de batterie, budget).

2. **Le Polymorphisme** : Permet de traiter différents types de voitures de manière uniforme dans les boucles de rendu.



### 3. LOGIQUE DE SIMULATION (ZONE 1 : RECHARGE) :

La première partie de la simulation gère l'arrivée des véhicules et la logique de recharge.

#### **Machine à États (State Machine)**

Chaque voiture (ChargingCar) suit une machine à états finis définie par l'énumération CarState. Cela permet de gérer des comportements complexes de manière organisée :

- ON\_MAIN\_ROAD : La voiture roule sur la route principale.
- ENTERING : Décision prise d'entrer dans la station (si batterie < 50%).
- WAITING : File d'attente si tous les chargeurs sont occupés.
- CHARGING : La voiture est connectée et remplit sa batterie.



- EXITING : Départ immédiat une fois la charge terminée.

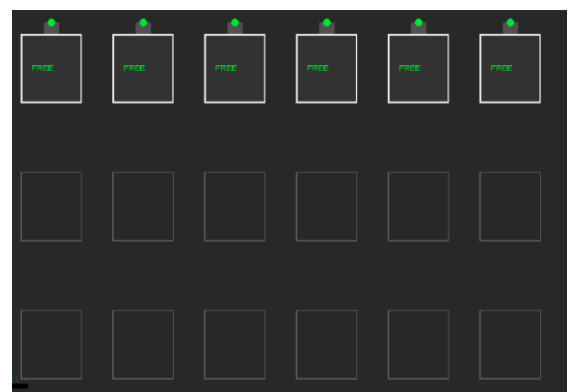
#### **Algorithme de File d'Attente**

La station utilise un système de priorité.

Lorsqu'une voiture entre, l'algorithme vérifie :

1. Si un slot de charge (ChargingSlot) est libre.
2. Sinon, elle est dirigée vers la file d'attente 1 (waitingRow1).
3. Si la file 1 est pleine, elle va vers la file 2 (waitingRow2).

Dès qu'un slot se libère, la voiture en tête de file avance automatiquement.



---

## 4. LE "HANDOFF" : TRANSITION D'ÉTATS :

L'innovation majeure de ce projet réside dans le "**Handoff Bridge**" (Le pont de transition).

### **Le Problème**

Dans la simulation, la Zone 1 (Recharge) et la Zone 2 (Parking) fonctionnent avec des logiques physiques et des classes différentes (ChargingCar vs ParkingCar).

### **La Solution**

Une ligne invisible sépare l'écran (Config::PARKING\_OFFSET\_X). Lorsqu'une voiture traverse cette ligne :

1. L'instance de ChargingCar est détruite.
2. Une nouvelle instance de SmartParkingCar est créée instantanément à la même position.
3. **Transfert de Données** : Les informations critiques (Niveau de batterie, Budget du conducteur, Type de véhicule) sont copiées de l'ancienne instance vers la nouvelle.

Cela crée une illusion de continuité parfaite pour l'observateur, tout en permettant de changer la logique de code sous-jacente (passage d'une logique de file d'attente à une logique de recherche de place).

## 5. GESTION INTELLIGENTE (ZONE 2 : STATIONNEMENT) :

La seconde zone simule un parking payant complexe avec une logique économique.

### Algorithme d'Allocation des Places

La classe ParkingManager utilise un algorithme de décision basé sur le budget:

- Le parking est divisé en zones tarifaires : 30, 20, 10 et 5
- Chaque voiture possède un **Budget** généré aléatoirement.
- L'algorithme cherche la *meilleure place disponible* que le conducteur peut se permettre.





---

## 6. FONCTIONNALITÉS TECHNIQUES PRINCIPALES

### 1. Architecture Hybride et Transfert de Données ("The Handoff")

- **Ségrégation des Classes** : La zone de recharge utilise la classe ChargingCar (optimisée pour la gestion de file d'attente et d'états), tandis que la zone de parking utilise ParkingCar (optimisée pour la physique de conduite et le stationnement).
- **Persistance des Données** : Lors du franchissement de la ligne de démarcation (Config::PARKING\_OFFSET\_X), un "pont" logiciel capture les attributs de l'instance ChargingCar (Niveau de batterie restant, Budget, Type de véhicule) et les injecte dans le constructeur de la nouvelle instance SmartParkingCar. Cela garantit une continuité logique pour l'utilisateur

### 2. Intelligence Artificielle et Pathfinding (Navigation)

Les véhicules ne se téléportent pas ; ils naviguent de manière autonome en suivant des trajectoires calculées dynamiquement.

- **Système de Waypoints (Points de passage)** : Chaque voiture possède une file d'attente de vecteurs (std::deque<Vector2> path). La voiture se dirige vers le premier point, et une fois atteint (distance < seuil), elle passe au suivant.
- **Calcul Vectoriel et Rotation** : L'orientation de la voiture est calculée en temps réel à l'aide de fonctions trigonométriques (atan2) basées sur la différence entre la position actuelle et la cible.
- **Lissage des Virages** : Pour éviter des rotations brusques, un algorithme de lissage (GetAngleDiff) limite la vitesse de rotation (Config::TURN\_SPEED), créant des courbes naturelles lors des manœuvres.

---

### 3. Algorithme d'Allocation de Ressources (Smart Parking)

Le gestionnaire de parking (ParkingManager) n'attribue pas les places au hasard. Il utilise un algorithme de filtrage basé sur des contraintes économiques :

1. **Analyse du Budget :** L'algorithme filtre d'abord les catégories de places (Premium, Standard, Éco) accessibles selon le budget de la voiture.
2. **Vecteurs de Priorité :** Des listes de priorité prédéfinies déterminent l'ordre de remplissage (par exemple, remplir les places proches de l'entrée en premier pour une gamme de prix donnée).
3. **Réservation Atomique :** Dès qu'une place est identifiée, elle est marquée comme "Réservée" (reserved = true) avant même que la voiture n'y arrive, empêchant d'autres véhicules de viser la même place

### 4. Machine à États Finis (FSM - Finite State Machine)

La gestion comportementale des véhicules dans la station de recharge repose sur une machine à états rigoureuse (enum CarState).

Cela permet de séquencer logiquement les actions : Approche -> Entrée -> Attente -> Connexion -> Charge -> Sortie.

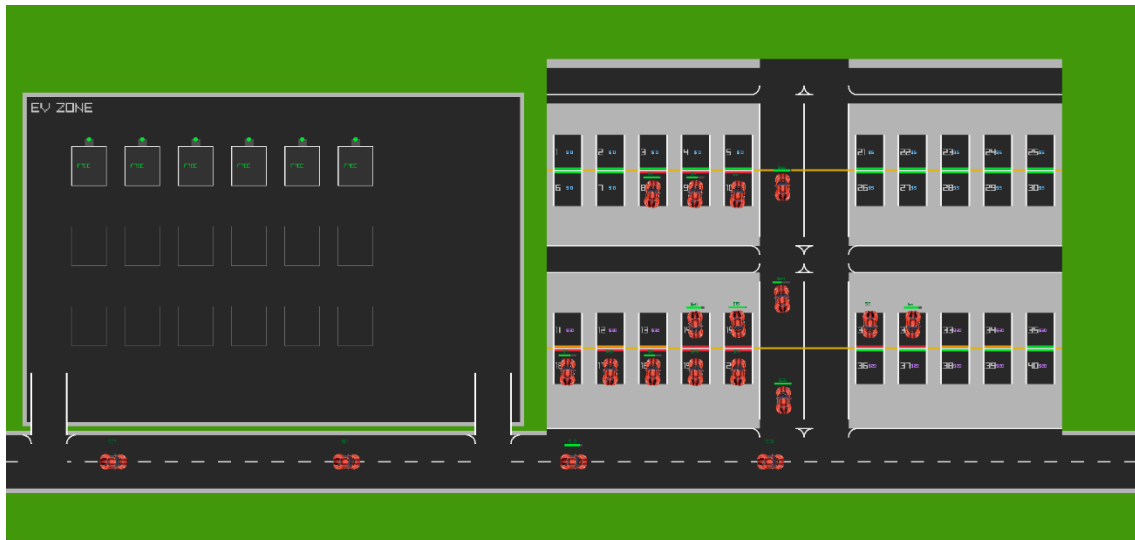
Cette structure évite les comportements erratiques et facilite le débogage.

### 5. Caméra Dynamique & Plein Écran

La simulation intègre une caméra 2D (Camera2D de Raylib) qui permet :

- Le Zoom (Molette de la souris) pour observer les détails

Avec Zoom out :



## Gestion Visuelle des Données

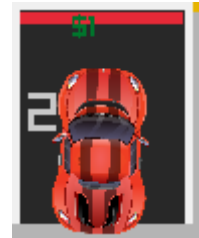
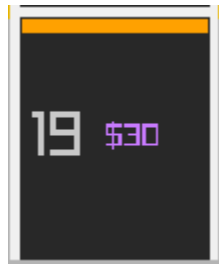
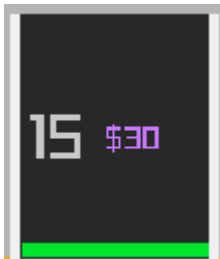
Contrairement à une simple console, toutes les données sont visualisées en temps réel :

- **Jauges de Batterie** : Barres vertes/rouges ou jaune au-dessus des voitures.



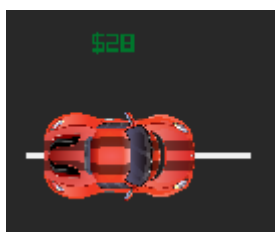
- **Indicateurs de Budget** : Affichage du montant (\$) au-dessus des véhicules.

- **État des Places** : Codes couleurs pour les places de parking (Vert = Libre, Rouge = Occupé, Orange = Réserve par une voiture en approche).

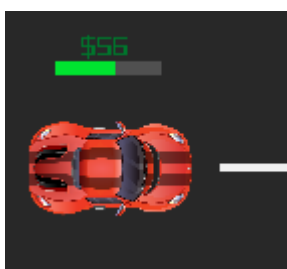
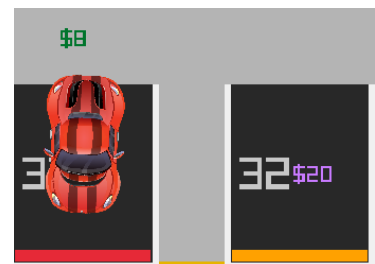


## Simulation Économique

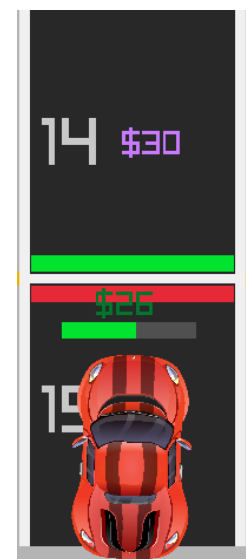
Les voitures "payent" virtuellement leur stationnement. Si le budget est suffisant, la transaction est validée. Cela ajoute une couche de réalisme économique à la simulation physique.



Après entrer au parking de 20\$



Après entrer au parking de 30\$



## 6. Le Système Anti-Collision : La "Zone de Sécurité"

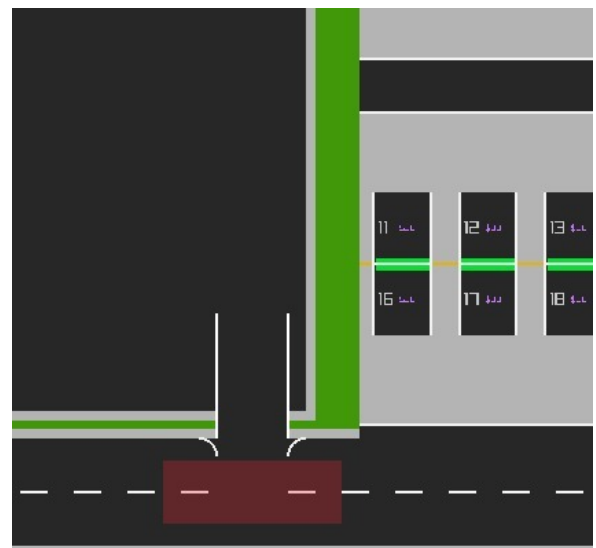
Pour éviter les accidents dans la simulation, nous avons implémenté un système de détection prédictive

Ce mécanisme fonctionne comme un capteur virtuel :

- **Projection de la Zone** : Chaque voiture projette une "boîte de sécurité" (Safety Box) devant elle, calculée en fonction de sa position, de sa taille et de sa direction actuelle.
- **Détection d'Obstacles** : À chaque image (frame), l'algorithme vérifie si cette boîte rouge entre en intersection avec la boîte d'une autre voiture ou d'un mur.
- **Réaction Immédiate (Freinage)** :
  - Si la boîte est libre : La voiture accélère ou maintient sa vitesse.
  - Si la boîte touche un obstacle : La voiture active un freinage d'urgence

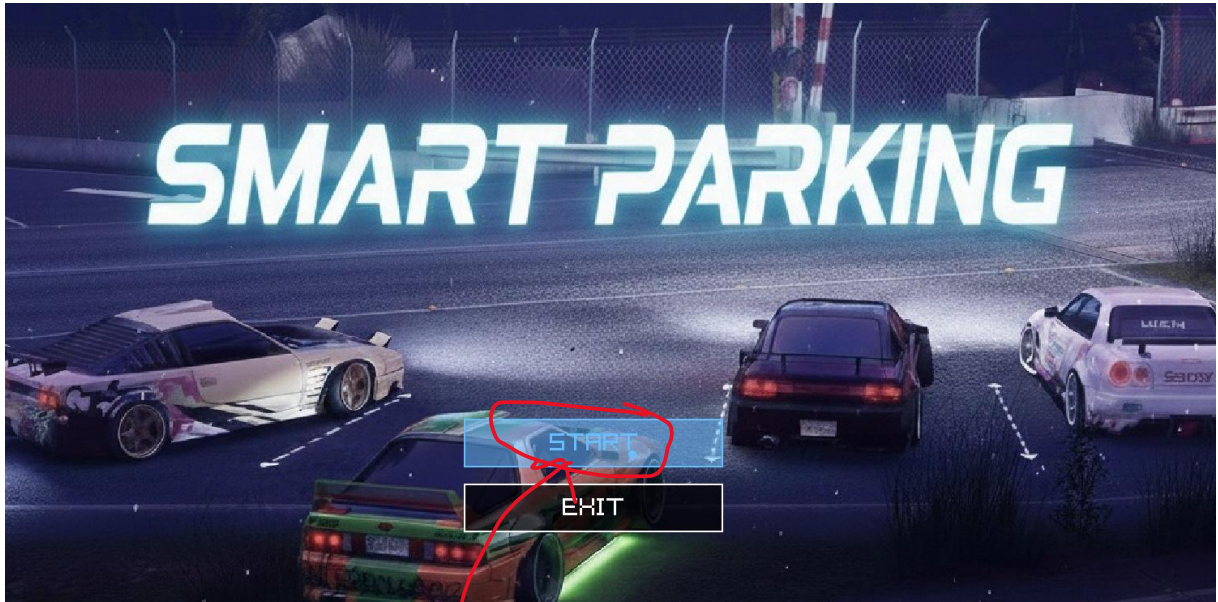
## 7. Invisible box :

Le transfert des véhicules entre la station de recharge et le parking est assuré par une "**Invisible Box**" (zone de déclenchement logique). Lorsqu'un véhicule franchit ce seuil virtuel, le système active le protocole de **Handover** : l'instance du véhicule est transférée d'un module à l'autre en conservant l'intégralité de ses attributs (niveau de batterie, budget, ID). Ce mécanisme garantit la persistance des données et offre une continuité visuelle fluide sans interruption pour l'utilisateur.

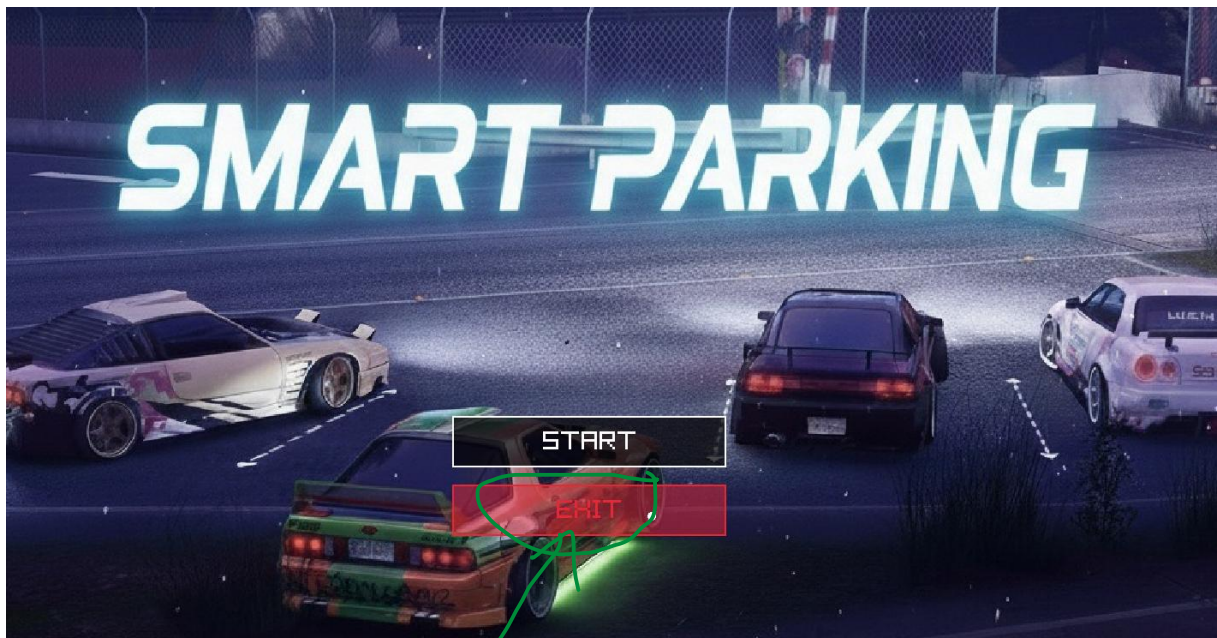


## 7. Captures d'écran de la simulation:

### Menu Principale :



Commence la simulation



sortir

---

## 8.CONCLUSION :

Ce projet de simulation a permis de mettre en pratique des concepts avancés de programmation C++ appliqués à une problématique concrète de ville intelligente.

Nous avons réussi à :

1. Créer une architecture **modulaire et évolutive**.
2. Simuler une **intelligence artificielle basique** (prise de décision autonome des voitures).
3. Gérer la **persistance des données** à travers différents systèmes (le système de Handoff).